



VIETNAM NATIONAL UNIVERSITY, HCMC

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Design and Implementation of the Hybrid FTP Application

Topic: Project 1: Technical Report

Course: Internetworking Protocol

Students:

Tran Duc An (23125024)

Le Hoang An (23125025)

Instructor:

Prof. Dung Tran Trung

Table of Contents

1	Application Scenario & Protocol Interaction	1
1.1	Sequence Diagram	1
2	Project-Wide Data Structures	2
2.1	TCP Control Packet Format	2
2.2	UDP Custom Header Fields (RDT)	2
2.3	Session Management Structures	2
3	Functional Workflows (Flowcharts)	3
3.1	Server Thread-Dispatch Logic	3
3.2	Reliable UDP (RDT) State Machine	4
3.3	Active vs. Passive Mode Toggle	5
4	Task Assignment Matrix	5
5	Self-Assessment & Peer Evaluation	6
6	Application Demo Evidence	6
A	GenAI Usage & Code Refinement Log	8
A.1	Prompt 1: Fixing Passive Mode Bug	8
A.2	Prompt 2: Path Traversal Vulnerability	8
A.3	Prompt 3: Explaining the Security Implementation	8

List of Tables

1	Custom Reliable UDP Packet Structure	2
2	Task Assignment Matrix	5
3	Peer Evaluation	6

List of Figures

1	Full TCP + UDP Lifecycle Sequence Diagram	1
2	Thread Dispatch Flowchart	4
3	RDT Sender/Receiver State Machine	5
4	Successful Upload (STOR) and Download (RETR)	6
5	End-to-End Hash Verification (HASH command)	7
6	Server Log: Connected Client Table and Concurrent Sessions	7

1 Application Scenario & Protocol Interaction

This section illustrates the full TCP and UDP lifecycle of the Hybrid FTP Application. The system decouples the control plane (TCP) from the data plane (UDP).

1.1 Sequence Diagram

The following sequence diagram demonstrates a standard interaction where a client authenticates, requests a passive data connection, and downloads a file via the custom Reliable Data Transfer (RDT) protocol over UDP.

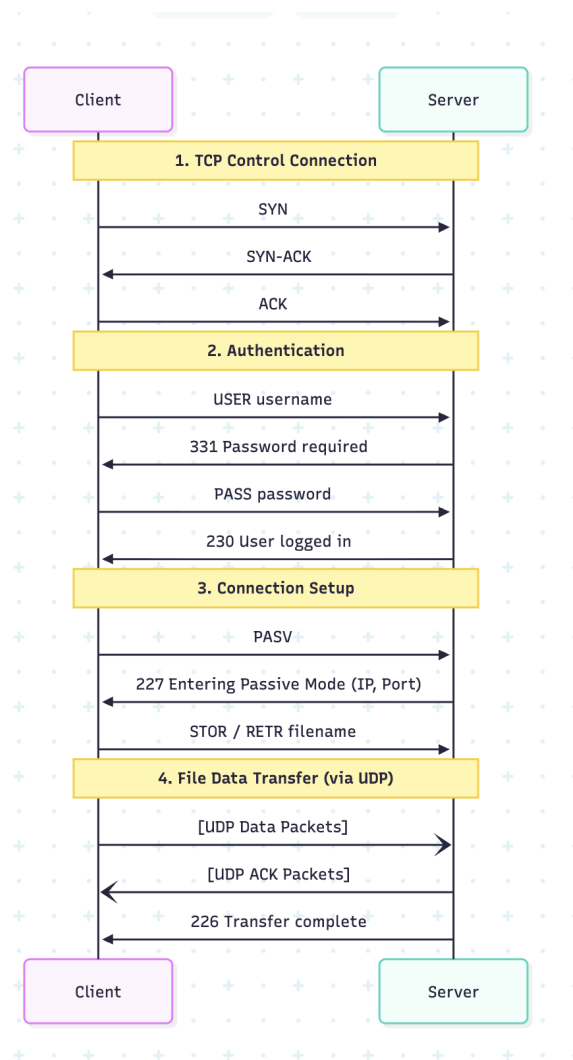


Figure 1: Full TCP + UDP Lifecycle Sequence Diagram

2 Project-Wide Data Structures

2.1 TCP Control Packet Format

The TCP control channel strictly transmits ASCII text. Commands follow the format `COMMAND [ARGUMENT]\r\n`. Server replies follow the format `STATUS_CODE Message\r\n`.

2.2 UDP Custom Header Fields (RDT)

To achieve reliability over UDP, data is encapsulated with a custom application-layer header before transmission. The structure is as follows:

Field	Size	Description
Sequence No.	4 bytes	Tracks packet ordering and duplication.
ACK No.	4 bytes	Acknowledges received packets.
Flags	1 byte	Control flags (e.g., SYN, FIN, ACK).
Checksum	32 bytes	SHA-256 / MD5 hash to detect payload corruption.
Payload Length	2 bytes	Specifies the size of the actual file chunk.
Payload	Variable	The actual file binary/text data chunk.

Table 1: Custom Reliable UDP Packet Structure

2.3 Session Management Structures

The server uses a thread-per-client model. Each thread maintains an isolated session state:

- `current_dir`: The sandboxed working directory for the client.
- `is_authenticated`: Boolean flag restricting access until `USER` and `PASS` are provided.
- `is_pasv`: Boolean flag indicating if the data channel is in Passive mode.
- `data_socket` / `client_data_addr`: References to the active UDP sockets and target addresses.
- `rename_from_path`: Temporarily caches a file path during the two-step `RNFR/RNTO` transaction.

3 Functional Workflows (Flowcharts)

3.1 Server Thread-Dispatch Logic

The main server process binds to TCP port 2121. Upon accepting a connection via `server_socket.accept()`, it spawns a dedicated daemon `threading.Thread`, passing the client socket to isolated handler logic, ensuring concurrency.

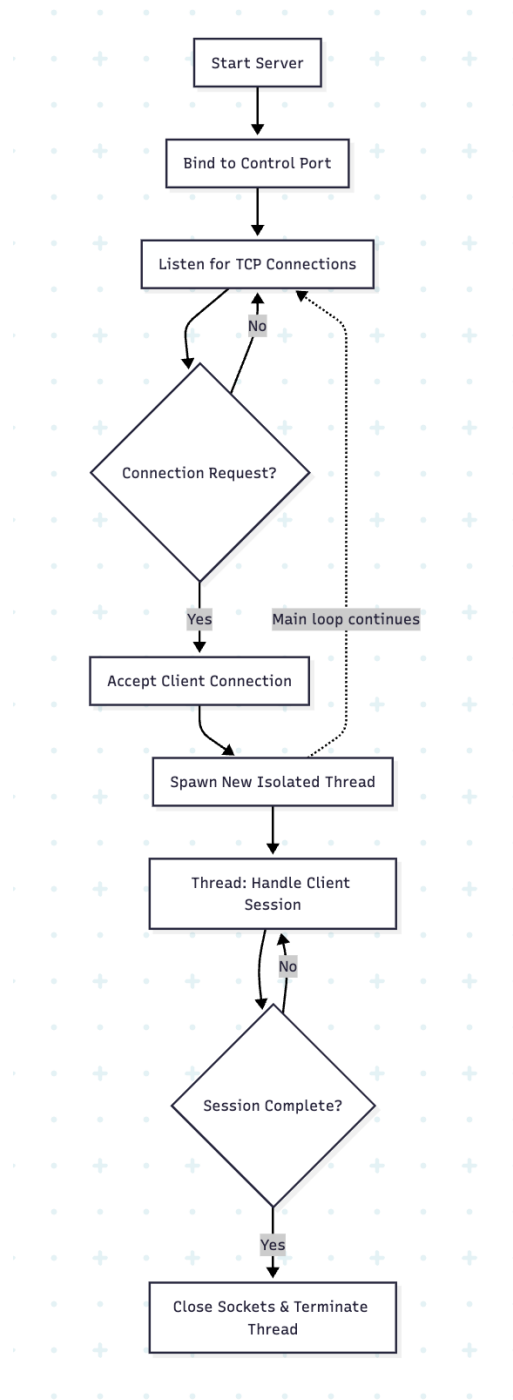


Figure 2: Thread Dispatch Flowchart

3.2 Reliable UDP (RDT) State Machine

File chunks are transmitted using Go-Back-N (GBN). The sender maintains a sliding window of unacknowledged packets. If a timeout occurs before receiving an ACK, the window retransmits.

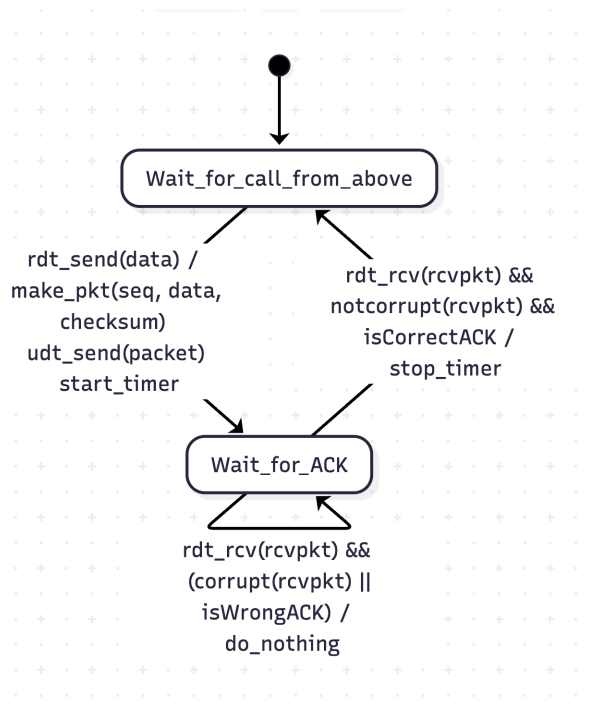


Figure 3: RDT Sender/Receiver State Machine

3.3 Active vs. Passive Mode Toggle

- **Active (PORT):** Client binds a UDP port, sends `PORT`, server saves the IP/Port. Server initiates the UDP knock to send/receive data.
- **Passive (PASV):** Server binds a random UDP port, sends `227 Entering Passive Mode`, sets `is_pasv = True`. Client initiates the UDP knock.

4 Task Assignment Matrix

Component / Module	Owner	Collaborator
TCP Control Channel & Auth	Student 1	Student 2
Reliable UDP (RDT) Protocol	Student 2	Student 1
Active/Passive Mode Logic	Student 1	Student 2
Extended FTP Commands (RNTO, STOU)	Student 2	Student 1
Security (Path Traversal Prevention)	Student 1	Student 2

Table 2: Task Assignment Matrix

5 Self-Assessment & Peer Evaluation

Both members actively researched protocol behaviors, collaborated on debugging the RDT sliding window, and paired programmed the concurrent server integration.

Member	Evaluation	Contribution
Student 1	Excellent communication, delivered TCP logic on time.	50%
Student 2	Engineered the robust RDT layer effectively.	50%

Table 3: Peer Evaluation

6 Application Demo Evidence

Note to examiners: The server logs and screenshots proving the functionality of the system are presented below.

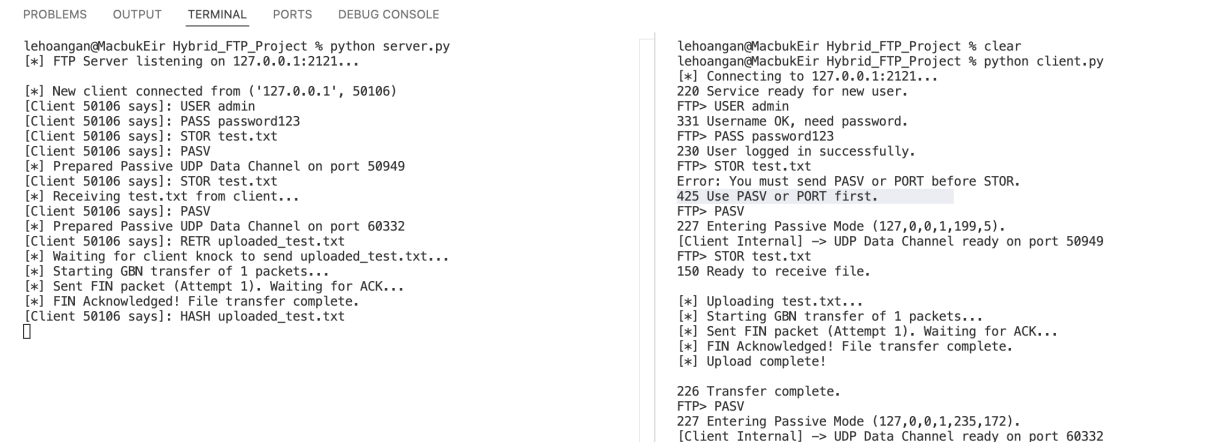


Figure 4: Successful Upload (STOR) and Download (RETR)

PROBLEMS	OUTPUT	TERMINAL	PORTS	DEBUG CONSOLE
		<pre> lehoangan@MacbukEir Hybrid_FTP_Project % python server.py [*] FTP Server listening on 127.0.0.1:2121... [*] New client connected from ('127.0.0.1', 50106) [Client 50106 says]: USER admin [Client 50106 says]: PASS password123 [Client 50106 says]: STOR test.txt [Client 50106 says]: PASV [*] Prepared Passive UDP Data Channel on port 50949 [Client 50106 says]: STOR test.txt [*] Receiving test.txt from client... [Client 50106 says]: PASV [*] Prepared Passive UDP Data Channel on port 60332 [Client 50106 says]: RETR uploaded_test.txt [*] Waiting for client knock to send uploaded_test.txt... [*] Starting GBN transfer of 1 packets... [*] Sent FIN packet (Attempt 1). Waiting for ACK... [*] FIN Acknowledged! File transfer complete. [Client 50106 says]: HASH uploaded_test.txt </pre>		<pre> 227 Entering Passive Mode (127,0,0,1,199,5). [Client Internal] -> UDP Data Channel ready on port 50949 FTP> STOR test.txt 150 Ready to receive file. [*] Uploading test.txt... [*] Starting GBN transfer of 1 packets... [*] Sent FIN packet (Attempt 1). Waiting for ACK... [*] FIN Acknowledged! File transfer complete. [*] Upload complete! 226 Transfer complete. FTP> PASV 227 Entering Passive Mode (127,0,0,1,235,172). [Client Internal] -> UDP Data Channel ready on port 60332 FTP> RETR uploaded_test.txt 150 Opening data connection for file transfer. [*] Downloading uploaded_test.txt... [*] Successfully saved as downloaded_uploaded_test.txt 226 Transfer complete. [*] Requesting server hash for integrity check... Server SHA-256: 1912766d6ba0e50e8b1bacfb51207e83b95b7ac0cd8ce15307cdf4965e7e3f6c Local SHA-256: 1912766d6ba0e50e8b1bacfb51207e83b95b7ac0cd8ce15307cdf4965e7e3f6c [+] VERIFIED: File transferred perfectly without corruption! FTP> </pre>

Figure 5: End-to-End Hash Verification (HASH command)

PROBLEMS	OUTPUT	TERMINAL	PORTS	DEBUG CONSOLE
		<pre> [*] FTP Server listening on 127.0.0.1:2121... [*] New client connected from ('127.0.0.1', 50106) [Client 50106 says]: USER admin [Client 50106 says]: PASS password123 [Client 50106 says]: STOR test.txt [Client 50106 says]: PASV [*] Prepared Passive UDP Data Channel on port 50949 [Client 50106 says]: STOR test.txt [*] Receiving test.txt from client... [Client 50106 says]: PASV [*] Prepared Passive UDP Data Channel on port 60332 [Client 50106 says]: RETR uploaded_test.txt [*] Waiting for client knock to send uploaded_test.txt... [*] Starting GBN transfer of 1 packets... [*] Sent FIN packet (Attempt 1). Waiting for ACK... [*] FIN Acknowledged! File transfer complete. [Client 50106 says]: HASH uploaded_test.txt [*] New client connected from ('127.0.0.1', 50136) [Client 50136 says]: USER admin [Client 50136 says]: PASS password123 [Client 50136 says]: PWD </pre>		<pre> t.py [*] Connecting to 127.0.0.1:2121... 220 Service ready for new user. FTP> USER admin 331 Username OK, need password. FTP> PASS password123 230 User logged in successfully. FTP> PWD 257 "/Users/lehoangan/Documents/GitHub/Hybrid_FTP_Pro ject" is current directory. FTP> </pre>

Figure 6: Server Log: Connected Client Table and Concurrent Sessions

A GenAI Usage & Code Refinement Log

In accordance with the academic integrity policy, this appendix strictly documents the usage of Generative AI tools (Google Gemini) used during development.

A.1 Prompt 1: Fixing Passive Mode Bug

Prompt: *"Check client.py and server.py. Do they accomplish all in the pdf file? Fix the PASV bug."*

Raw AI Output: The AI identified that `is_pasv` was never being set to `True` when the PASV command was received, causing all transfers to crash by falling back to Active mode without an address.

Refinement & Critical Analysis: We accepted the AI's logic to inject `is_pasv = True` into the PASV block. However, we manually audited the codebase to ensure that `is_pasv` is correctly reset to `False` after a successful file transfer or an `ABOR` command to comply with the standard one-shot behavior of FTP passive modes.

A.2 Prompt 2: Path Traversal Vulnerability

Prompt: *"Fix security issue where CWD / escapes the directory."*

Raw AI Output: The AI generated a helper function `get_safe_path(base_dir, current_dir, target_path)` using `os.path.abspath` and `startswith()` to sandbox operations.

Refinement & Critical Analysis: We reviewed the AI's implementation and manually propagated this wrapper function to all file operation commands (`STOR`, `RETR`, `RNFR`, `DELE`, `MKD`) instead of just `CWD` to ensure complete filesystem security across the entire server.

A.3 Prompt 3: Explaining the Security Implementation

Prompt: *"Explain how the get_safe_path function from Prompt 2 actually works to prevent path traversal."*

Raw AI Output: The AI explained that the function first normalizes the user's requested path using `os.path.abspath()`, which mathematically resolves any relative directory jumps like `../` or `./`. After computing the absolute path, it uses the `startswith()` method to verify that the resulting

path string begins exactly with the server's designated root directory. If a user tries to access a path outside the root, `startswith()` returns false, and the action is blocked.

Refinement & Critical Analysis: This explanation deepened our understanding of directory traversal vulnerabilities at the OS level. By grasping the underlying mechanism, we were able to ensure that this validation check was rigorously applied as a centralized middleware layer before any file descriptors are opened in the server logic, rather than relying on ad-hoc checks.